

AAKAAR — Backend Architecture (v1)

Two FastAPI services, one shared philosophy: every row is tenant-scoped, every capability is registered, every run is auditable. This document covers the request lifecycle, the database, the run lifecycle, and the operational concerns.

1. Two backends

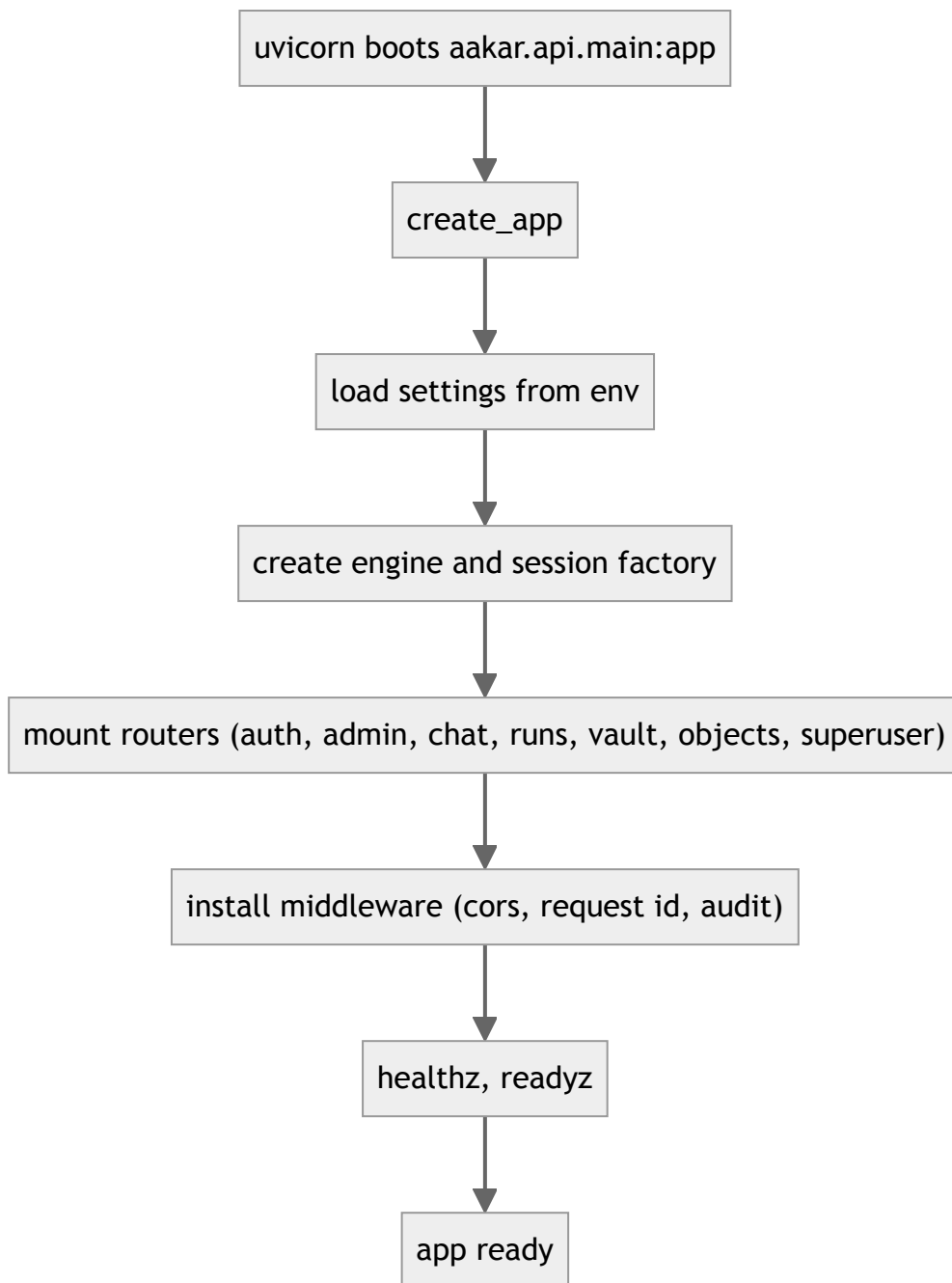
Service	Path	Port (dev)	Purpose
aakar API	aakar/aakar/api	8000	Tenant traffic and superuser surfaces: auth, chat, runs, vault, objects, admin.
admin-app API	admin-app/server	8001	Demo fixture — a mock bank-ops service used as a third-party site during demos. Not the superuser surface.

Both speak FastAPI + Pydantic v2 + SQLAlchemy 2. They share migration tooling (Alembic) but maintain separate schemas in v1. The split is intentional: tenant traffic must not contend with demo traffic, and the admin-app's UI labels are part of the planner's selector-targeting heuristics.

2. Stack

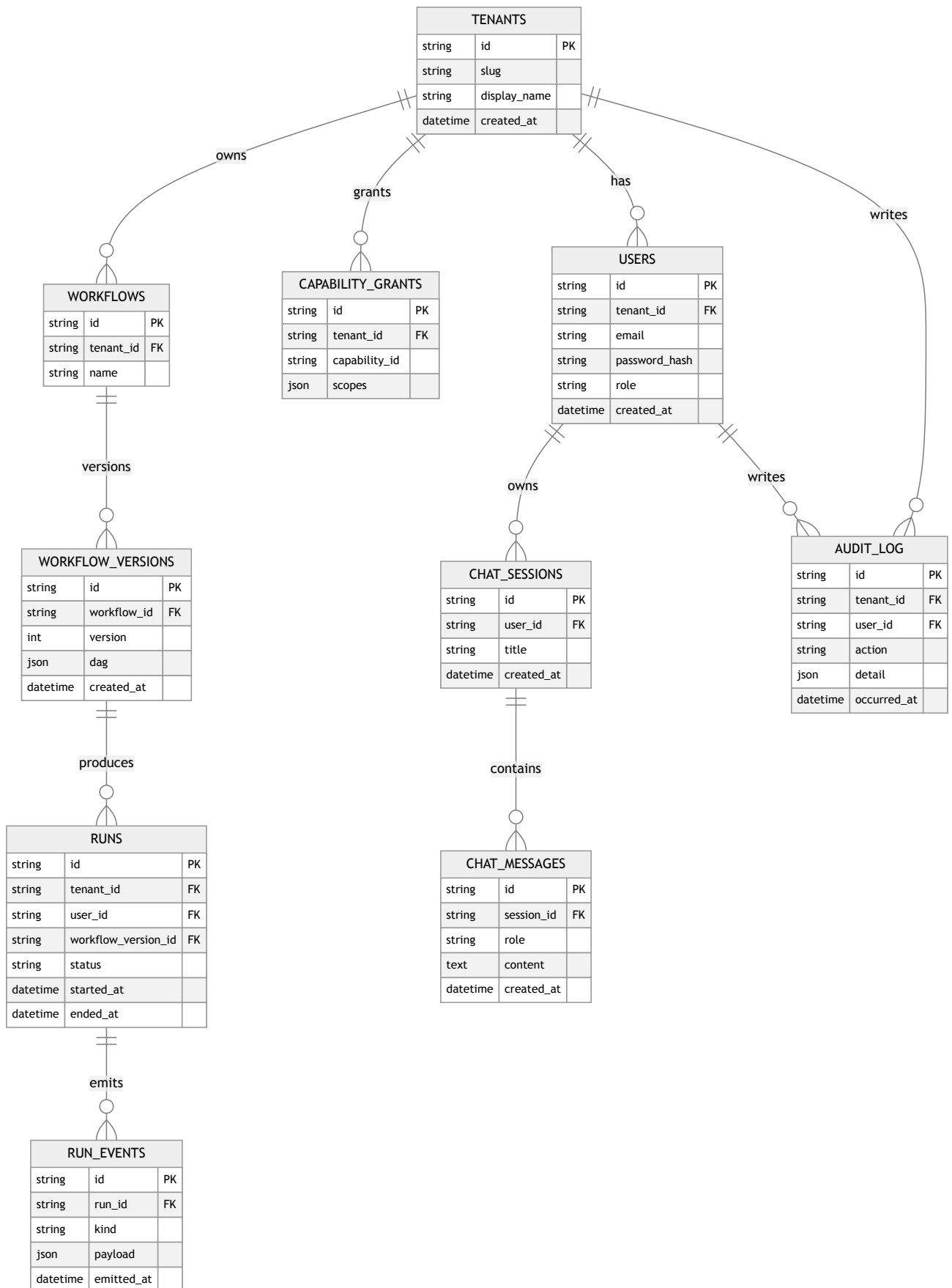
Concern	v1 choice
Language	Python 3.12
Web framework	FastAPI
Validation	Pydantic v2
ORM	SQLAlchemy 2
Migrations	Alembic
Local DB	SQLite
Cloud DB	Yugabyte (Postgres wire)
Vector index	FAISS (SQLite mode), pgvector (planned for Yugabyte mode)
Object store	Filesystem under <code>AAKAR_DATA_DIR</code>
Browser worker	Playwright headless Chromium (1.59)
HTTP worker	httpx
Auth	bcrypt + HS256 JWT
LLM	OpenAI Chat Completions strict JSON
Embeddings	BGE small (<code>BAAI/bge-small-en-v1.5</code>) via sentence-transformers
Background work	asyncio + a small threadpool

3. App factory



The factory pattern keeps tests cheap: each integration test builds a fresh app bound to a temp SQLite file.

4. Database schema

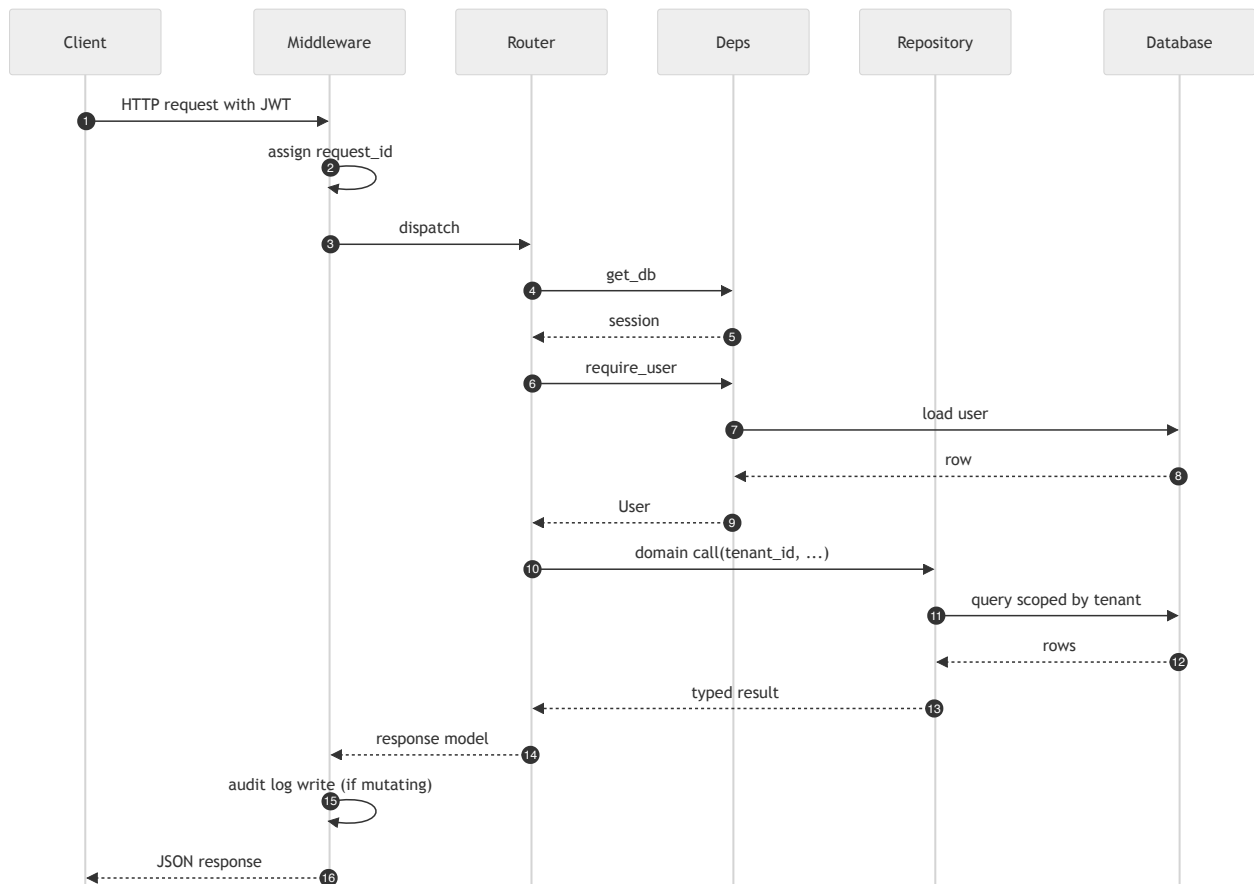


Conventions:

- All ids are ULIDs encoded as 26-char strings.

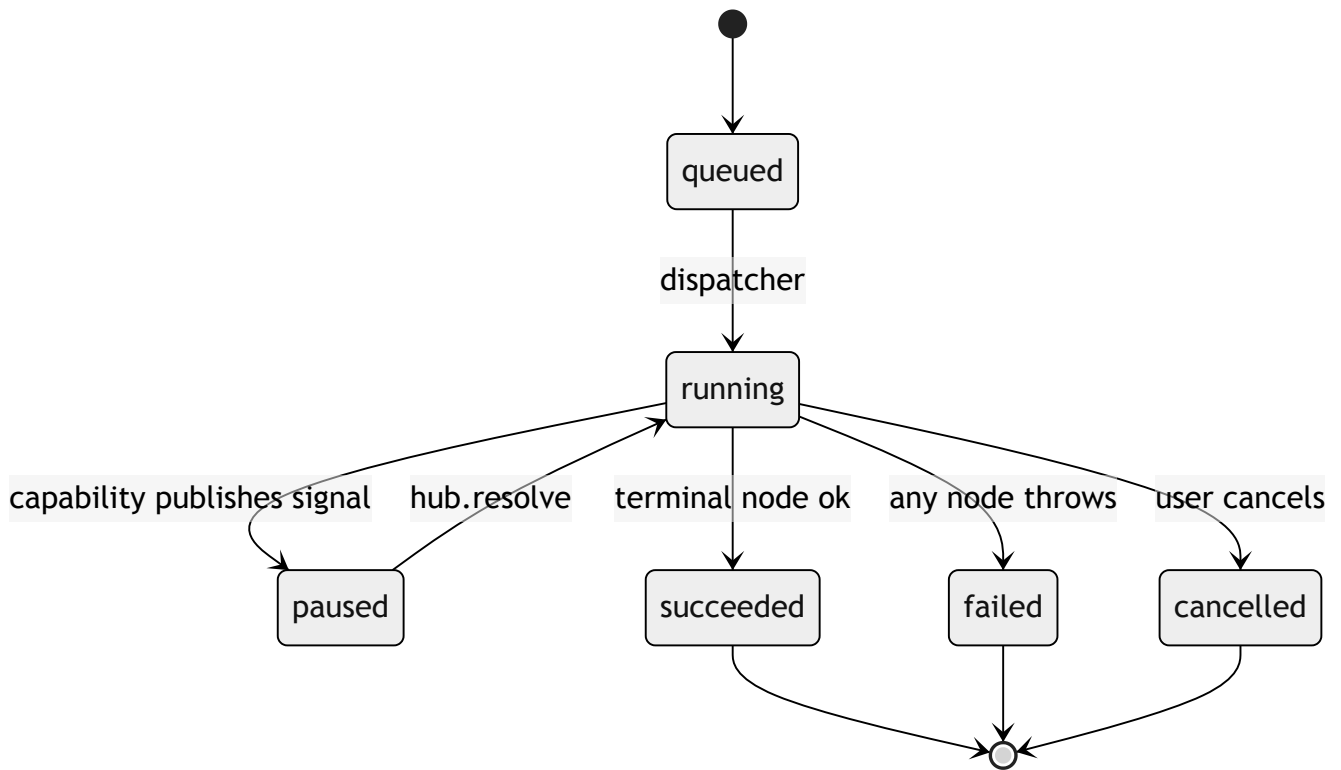
- Every tenant-bearing table has a unique index on `(tenant_id, ...)` first; Postgres planner uses it.
- `RUN_EVENTS.payload` is JSON; PII fields are scrubbed before write.
- `AUDIT_LOG` is append-only; deletes are forbidden by an Alembic check.

5. Request lifecycle



Tenant scoping is enforced inside repositories. Routers cannot accidentally widen the scope because repositories require a `tenant_id` argument and ignore any free-form filter that would broaden it.

6. Run lifecycle

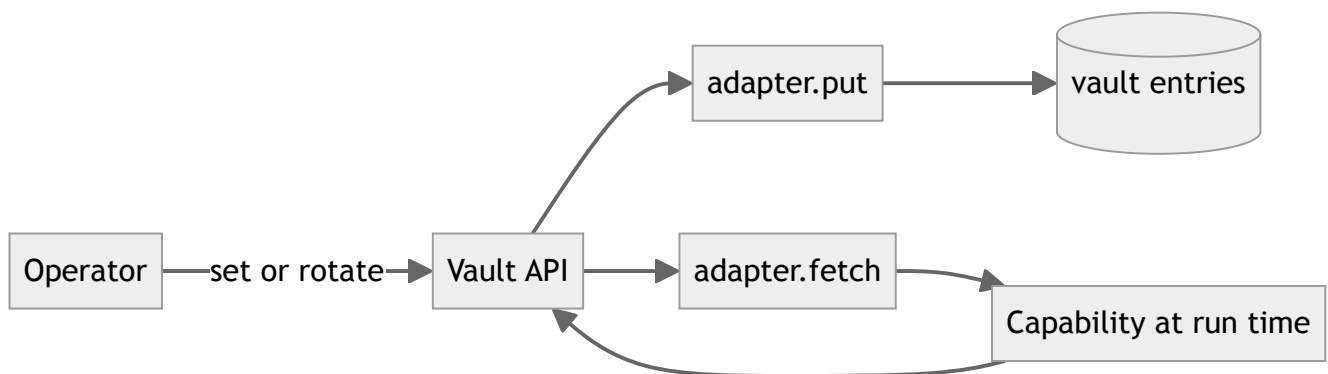


A retry of a failed run creates a new `RUNS` row with a `parent_run_id` reference. Audit history is preserved.

7. Vault

The vault stores per-tenant credentials. Each entry binds:

- `tenant_id`
- `site_id` (the third party, for example `nbbl` or `hdfc`)
- `user_handle` (the operator's identity on that site)
- `secret_blob` (storage form depends on adapter — see below)
- `metadata` (rotation hints, last-used)



v1 honesty: the local adapter writes JSON files at `{data}/vault/{tenant_id}/{ref}.json` with mode 0600. There is **no cipher at rest** in v1. A KMS-backed adapter (encrypt-on-write, decrypt-at-

read, per-tenant data key wrapped by a master key in a managed KMS) is the Phase 2 roadmap item — the code path stays the same, only the adapter changes.

8. Object storage

Artifacts (downloaded files, screenshots, run-time uploads) are persisted under:

```
$AAKAR_DATA_DIR/  
  tenants/  
    <tenant_id>/  
      runs/  
        <run_id>/  
          events/  
            <event_id>.png  
          downloads/  
            <filename>  
          uploads/  
            <filename>
```

Each row that references an artifact stores its URI as `aakar://t/<tenant>/runs/<run>/...` in v1. Switching to S3 or GCS is one adapter implementation away; row-level URIs already encode the namespace.

9. FAISS capability index

The capability index is built at process start from the registered capability set. It supports:

- top-k similarity search by prompt embedding
- metadata filter by tenant grants
- rebuild on capability registration / grant change

The index is a single FAISS `IndexFlatIP` plus a parallel sidecar list of capability ids, partitioned per tenant. For Yugabyte deployments the same data lives in a `pgvector` table and the planner uses a SQL ANN query instead (planned, not shipped in v1).

Status: v1 maintains the index but the planner does not yet read from it at plan-time. Wiring it into `PromptBuilder` for tenants with many grants is a Phase 1 follow-up.

10. Dashboard queries

The frontend dashboard pulls aggregates through three endpoints:

Endpoint	Returns
<code>/stats/runs/daily</code>	run counts per day, last 30 days, by status
<code>/stats/capabilities/usage</code>	top capabilities by run count
<code>/stats/sites/health</code>	per-site success rate over last 7 days

All three are materialized at query time from `RUNS` and `RUN_EVENTS` with simple `GROUP BY` aggregations. v1 makes no attempt at pre-aggregation.

11. Settings table

A small `settings` table holds per-tenant feature flags and tunables:

Key	Default	Notes
<code>planner.strategy</code>	<code>auto</code>	<code>auto</code> , <code>oneshot</code> , or <code>agentic</code> .
<code>executor.max_concurrent_runs</code>	<code>4</code>	Per-tenant cap on parallel runs.
<code>browser.timeout_ms</code>	<code>30000</code>	Default per-step timeout.
<code>vault.rotation_days</code>	<code>90</code>	Reminder cadence; non-blocking.

12. Bootstrap and seeding

On first boot, the API:

1. Runs Alembic migrations to head.
2. Seeds platform metadata: capability registry rows from code, control rows from YAML.
3. Optionally seeds a demo tenant with an admin user if `AAKAR_SEED_DEMO=1` .
4. Builds the FAISS index in memory from the seeded capabilities.

The seed step is idempotent. Re-running it does not duplicate rows. A superuser is seeded from `AAKAR_SUPERUSER_EMAIL` / `AAKAR_SUPERUSER_PASSWORD` if those env vars are present; v1 deployments may run with no superuser at all and bootstrap one out-of-band.

13. admin-app server

The admin-app's FastAPI service is intentionally thin and **not** the superuser surface:

- It owns the demo recon upload endpoint (`/api/recon/uploads`) and the upload history list — these mimic a real bank-ops backend that the planner targets.
- It does not access the tenant database. Superuser surfaces go through the aakar API with a superuser JWT.

- Uploaded recon files are stored on disk under `admin-app/server/uploads/` and referenced from an in-memory history table.

14. Operations runbook (v1, dev)

Task	Command
Start everything	<code>./start.sh</code>
Run migrations	<code>(cd aakar && .venv/bin/alembic upgrade head)</code>
Run backend tests	<code>(cd aakar && .venv/bin/pytest)</code>
Reset local DB	<code>rm aakar/data/aakar.sqlite && alembic upgrade head</code>
Tail aakar logs	follow stdout in the API Terminal window
Stop a service	Ctrl-C in its window or <code>kill.sh</code>

`start.sh` opens five Terminal windows: aakar API (8000), admin-app API (8001), aakar-web (5173), admin-app (3000), nbbl-app (3001). Numerical libs are pinned to single-thread mode in the env to avoid leaked-semaphore warnings on shutdown.

15. Failure modes

Scenario	Detection	Behavior
LLM returns malformed JSON	Pydantic parse	Retry once then surface error to chat.
LLM emits unknown capability	DAG validator	Reject DAG, return clarification.
Browser crashes mid-run	Heartbeat timeout	Mark run failed; preserve events; surface retry button.
Vault entry missing	Capability lookup	Pause with a <code>picker</code> signal asking the user to add credentials.
Disk full	Object write error	Mark run failed; alert via audit log.
Migration failure	Alembic	Refuse to start; print the failing revision.

16. Backups (v1, manual)

Local SQLite is in `aakar/data/aakar.sqlite`; an `sqlite3 .backup` run from cron is sufficient for dev. Yugabyte is backed up via the platform's standard snapshot tooling. Object store contents

(artifacts) are tarred per run when a run finishes; the tarball is sufficient evidence to replay UI state.

17. Reading guide

- For motivation and boundaries, read the HLD.
- For per-module details, read the LLD.
- For UI shape and live screen, read the frontend doc.
- For what comes next (including OpenTelemetry), read the roadmap.